Task 1 Documentation

Python File Handling, Automation and Exception Handling

Name: Harshitha Bolla

Course: B.Tech CSE, Anurag University

Internship: Alfido Tech - Python Internship

Task: Task 1 - File Organizer Automation Script

1. Objective

The objective of this task was to understand file handling, automation logic, and exception handling in Python. The script organizes files inside a folder automatically by sorting them into subfolders based on their file extension.

2. Requirements Covered

- Read and write files (txt/csv)
- Automate file operations - rename, move, delete
- Use try-except for error handling
- Explain code logic using comments

3. Tools Used

- Python 3
- os module
- shutil module

4. Project Structure

The task1 folder contains the script (organizer.py), a README file, and a test_folder which is used to test the script.

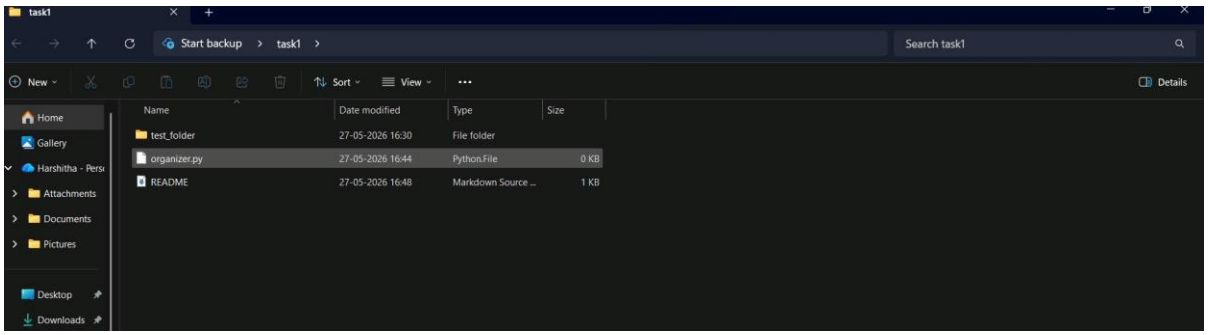

Figure 1: Project folder structure

5. Key Code Logic

The full script is available in the GitHub repository (link at the end of this document). Below are the main logic parts of the code.

Reading files and checking extension:

```
for file in os.listdir(source_folder):
    if os.path.isfile(file_path):
        extension = os.path.splitext(file)[1].lower()
```

Matching the extension to a category and moving the file:

```
for folder_name, extensions in file_types.items():
    if extension in extensions:
        os.makedirs(target_folder, exist_ok=True)
        shutil.move(file_path, os.path.join(target_folder, file))
        moved = True
```

Handling files that do not match any category:

```
if not moved:
    print(f"No category found for {file}")
```

6. Code Explanation

- **Setup:** The script imports os and shutil. A dictionary named file_types stores file categories and their related extensions.
- **Reading the Folder:** os.listdir() is used to read all items inside the source folder. os.path.isfile() checks whether the item is a file and not a folder.
- **Matching File Extension:** os.path.splitext() is used to get the extension of the file. The extension is converted to lowercase so that the comparison works correctly.
- **Moving the File:** If the extension matches a category, a new folder is created using os.makedirs() if it does not already exist. Then shutil.move() moves the file into that folder.
- **Files With No Matching Category:** If a file extension does not match any category, the script prints a message saying no category was found for that file.

7. Exception Handling

The script can be extended using try-except blocks around the file operations to avoid the program stopping if an error occurs. Some examples:

- FileNotFoundError - if the source folder does not exist
- PermissionError - if a file is open or access is denied
- shutil.Error - if there is an issue while moving the file

Using try-except here ensures that even if one file causes an error, the program can continue processing the remaining files instead of crashing.

8. Sample Input and Output

Before Running the Script: The test_folder contains different types of files mixed together before running the script.

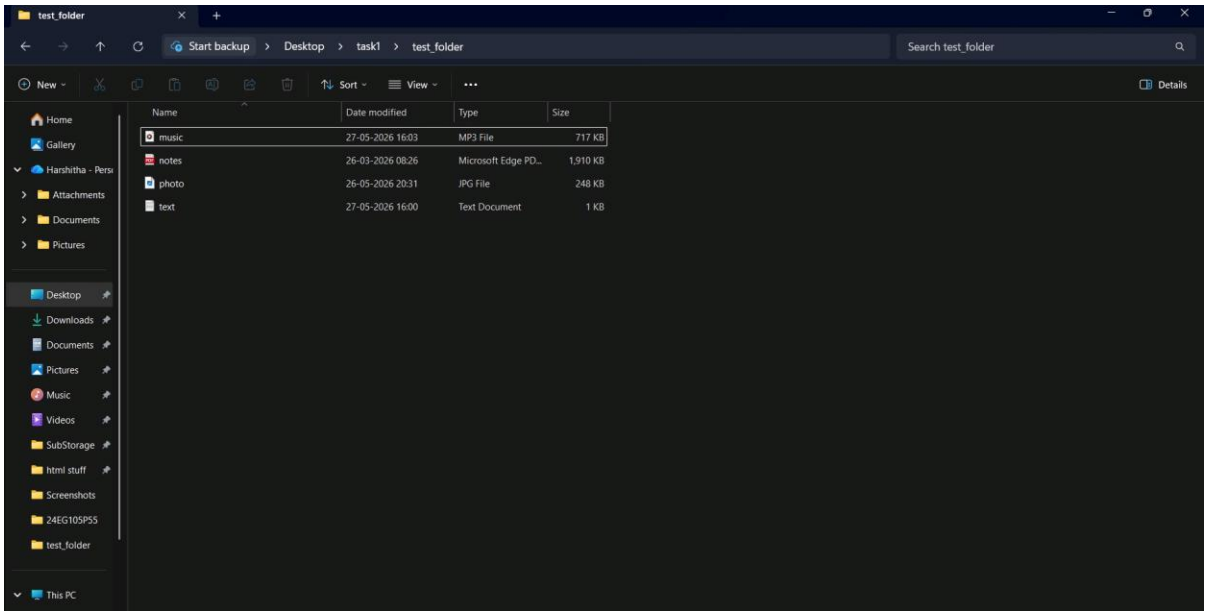

Figure 2: Folder before running organizer.py

After Running the Script: After running the script, the files are automatically sorted into folders (Audio, Images, PDFs, Text) based on their type.

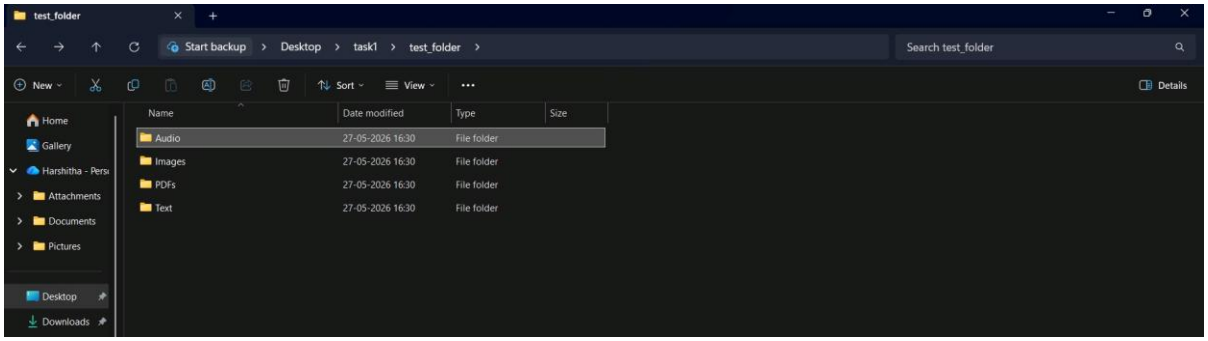

Figure 3: Folder after running organizer.py

9. Conclusion

This task helped in understanding how Python can be used to automate file organization. It covered reading folder contents, checking file types, creating folders automatically, and moving files. It also helped in understanding where exception handling should be added to make the script more reliable.

10. GitHub Repository

https://github.com/HarshithaBolla17/Alfido-Tech-Python-Internship